GATE CS 2027 SYLLABUS

Section 1: Engineering Mathematics

Discrete Mathematics: Propositional and first order logic. Sets, relations, functions, partial orders and lattices. Monoids, Groups. Graphs: connectivity, matching, colouring. Combinatorics: counting, recurrence relations, generating functions

Linear Algebra: Matrices, determinants, system of linear equations, eigenvalues and eigenvectors, LU decomposition

Calculus: Limits, continuity and differentiability, Maxima and minima, Mean value theorem, Integration

Probability and Statistics: Random variables, Uniform, normal, exponential, Poisson and binomial distributions. Mean, median, mode and standard deviation. Conditional probability and Bayes theorem

Section 2: Digital Logic

Boolean algebra and minimization – algebraic technique, Karnaugh map, tabular method. Design of combinational and sequential circuits. Number representation and arithmetic (fixed and floating point)

Section 3: Computer Organization and Architecture

Instruction set and addressing modes. Design of arithmetic and logic unit (ALU). Design of control unit – hardwired and microprogrammed.

Memory interfacing and hierarchy: performance, cache memory mapping.

I/O interface (interrupt and DMA). Instruction pipelining, pipeline hazards.

Section 4: Programming and Data Structures

Programming in C. Recursion. Arrays, stacks, queues, linked lists, trees, binary search trees, binary heaps, graphs

Section 5: Algorithms

Searching, sorting, hashing. Asymptotic worst case time and space complexity.

Algorithm design techniques: greedy, dynamic programming and divide-and-conquer. Graph traversals, minimum spanning trees, shortest paths.

Section 6: Theory of Computation

Regular expressions and finite automata. Context-free grammars and push-down automata. Regular and context-free languages, pumping lemma. Turing machines and undecidability.

Section 7: Compiler Design

Lexical analysis, parsing, syntax-directed translation. Runtime environments. Intermediate code generation.

Local optimisation, Data flow analyses: constant propagation, liveness analysis, common sub expression elimination.

Section 8: Operating System

System calls, processes, threads, inter-process communication, concurrency and synchronization. Deadlock. CPU and I/O scheduling. Memory management and virtual memory. File systems.

Section 9: Databases

ER-model. **Relational model:** relational algebra, tuple calculus, SQL. Integrity constraints, normal forms. File organization, indexing (e.g., B and B+ trees). Transactions and concurrency control.

Section 10: Computer Networks

Principles of Layering; Basics of switching (circuit, packet and virtual circuit) and performance metrics;

Data link layer: error detection, Medium Access Control, Ethernet; Distance vector and link state routing;

IPv4 - Fragmentation, CIDR Notation, Network Address Translation; TCP- flow control and congestion control, socket API; DNS and HTTP.

Tier 1

DSA 13M-15M

Trees, Graphs, DP, Sorting

EM 13M

Linear Algebra, Probability and Statistics

General Aptitude 15M

Operating Systems 8M-10M

Process Scheduling, Memory Management, Deadlocks

Tier 2

CN 7M-8M

TCP/IP Stack, Routing, Congestion Control

DBMS 7M-8M

ER Diagrams, SQL, Normalization, Transactions

COA 7M-8M

Cache, Pipeline, Memory Hierarchy

Tier 3

Theory of Computation 5M-7M

Compiler Design 4M-6M

Digital Logic 4M-5M

To achieve an AIR under 70, you need an execution plan that tracks metrics down to the daily block. Starting next week (Monday, October 12, 2026), you have exactly 7 weeks until November 30.

we are compressing your native strengths (Mathematics) into highly aggressive hybrid blocks, freeing up deeper focus for complex systems architecture (COA, OS) where non-CS backgrounds usually lose marks.

Here is your daily execution framework, followed by the week-by-week deep dive.

The Daily 9-Hour Micro-Schedule

To sustain this pace, split your day into three distinct cognitive blocks:

[Block 1: Deep Focus (3.5 Hours)] —> [Block 2: Active Problem Solving (3.5 Hours)] —> [Block 3: Test & Fix (2 Hours)]

• Block 1: Theory & Concept Mapping (3.5 Hours | Morning): Learn core architectures, algorithms, or systems properties. Use concise resources like GATE Overflow notes or rapid video lectures.

• Block 2: High-Intensity PYQ Sprint (3.5 Hours | Afternoon): Solve 25–30 targeted GATE questions from the last 25 years. Do not check solutions until you have attempted a question for at least 7 minutes.

• Block 3: The Revision & Error Log Block (2 Hours | Night): Review what you got wrong. Write the exact mathematical or conceptual reason for your failure in a physical Error Notebook.

The 7-Week Micro-Level Schedule

🗓️ Week 1 (Oct 12 – Oct 18): Discrete Math, Linear Algebra & Digital Logic

• Syllabus Focus: Mathematical Logic, Set Theory, Combinatorics, Graph Theory; Matrices, Eigenvalues, System of Linear Equations; K-Maps, Minimization (Tabular Method), Combinational Circuits (Mux, Decoders), Sequential Circuits (Flip-Flops, Counters).

• AIR 70 Execution Strategy:

• Mon–Wed: Exploit your math background. Skip video lectures entirely for Discrete Math and Linear Algebra. Go straight to GATE PYQs. Focus heavily on Graph Theory properties (Planar graphs, chromatic numbers) as GATE loves tricky theoretical MSQs here.

• Thu–Sat: Shift to Digital Logic. Master Sequential Circuit analysis (finding the counting sequence of arbitrary flip-flops). Ensure you practice the newly emphasized Quine-McCluskey (Tabular) minimization method.

• Sunday: Take 1 Discrete Math Subject Test and 1 Digital Logic Subject Test.

🗓️ Week 2 (Oct 19 – Oct 25): C Programming, Data Structures & Probability

• Syllabus Focus: Pointers, Parameter passing, Recursion; Arrays, Stacks, Queues, Linked Lists, Trees (BST, AVL, Binary Heaps); Conditional Probability, Bayes Theorem, Random Variables, Distributions (Binomial, Poisson, Normal).

• AIR 70 Execution Strategy:

• Mon–Tue: Focus strictly on C Pointers, Double Pointers, and Dynamic Memory Allocation. GATE frequently tests code snippets with complicated pointer arithmetic designed to make you miscount steps.

• Wed–Fri: Master Tree properties (e.g., maximum nodes in an AVL tree of height h). Practice drawing the intermediate states of a Max/Min Heap during insertion and deletion.

• Sat: Knock out Probability PYQs. Pay extreme attention to Exponential and Uniform distribution integrations.

• Sunday: Take a combined Programming & Data Structures subject test. Log code-tracing errors in your Error Notebook.

🗓️ Week 3 (Oct 26 – Nov 01): Algorithms & Calculus

• Syllabus Focus: Asymptotic Analysis, Recurrence Relations; Divide & Conquer, Greedy, Dynamic Programming (DP); Graph Traversals (BFS, DFS), MSTs (Prim, Kruskal), Shortest Paths (Dijkstra, Bellman-Ford); Limits, Continuity, Maxima/Minima.

• AIR 70 Execution Strategy:

• Mon–Tue: Master solving recurrences using Master's Theorem and substitution. Practice identifying optimal substructure and overlapping subproblems for core DP problems (Matrix Chain Multiplication, LCS).

• Wed–Fri: Focus on Graph Algorithms. Learn to trace Dijkstra and Bellman-Ford step-by-step. Understand when these algorithms fail (e.g., Dijkstra with negative edge weights).

• Sat: Review Calculus. GATE CS calculus is straightforward—focus heavily on finding Maxima and Minima using derivatives.

• Sunday: Take an Algorithms subject test.

🗓️ Week 4 (Nov 02 – Nov 08): Theory of Computation & Compiler Design

• Syllabus Focus: Regular Expressions, DFA/NFA minimization, Closure Properties, Context-Free Grammars, Turing Machines, Undecidability; Parsing (LL(1), LR(1), LALR), Syntax-Directed Translation (SDT).

• AIR 70 Execution Strategy:

• Mon–Wed: Treat TOC as formal set theory. Memorize the Closure Property Table completely (which languages are closed under intersection, complement, etc.). Practice proving undecidability using reduction lines.

• Thu–Sat: For Compilers, learn to construct LL(1) parsing tables (First & Follow sets) and LR(1) item sets. Do not skip Syntax-Directed Translations—GATE frequently asks you to calculate the final value printed by a bottom-up parser using semantic rules.

• Sunday: Take a full TOC + Compiler Design combined subject test.

🗓️ Week 5 (Nov 09 – Nov 15): Operating Systems (OS)

• Syllabus Focus: Process States, CPU Scheduling; Semaphores, Classical Synchronization Problems, Deadlocks; Paging, Multi-level Paging, Inverted Paging, Virtual Memory, Page Replacement Algorithms.

• AIR 70 Execution Strategy:

• Mon–Tue: CPU scheduling is an easy scoring area (Gantt charts). Spend extra hours on Process Synchronization. Practice decoding whether a given pseudocode block satisfies Mutual Exclusion and Bounded Waiting.

• Wed–Fri: Paging is purely mathematical. Master the formulas mapping Virtual Address space to Physical Address space. Practice calculations for Multi-level paging and TLB hit/miss ratios.

• Sat: Solve problems on Disk Scheduling (SCAN, C-SCAN) and Page Replacement (LRU, Optimal).

• Sunday: Take 2 OS Subject Tests.

🗓️ Week 6 (Nov 16 – Nov 22): Computer Organization & Architecture (COA)

• Syllabus Focus: Machine Instructions, Addressing Modes; Instruction Pipelining, Hazards, Speedup; Cache Memory Mapping (Direct, Associative, Set-Associative), Cache Write Policies.

• AIR 70 Execution Strategy:

• Mon–Tue: Master Addressing Modes (Indirect, Indexed, Auto-increment). Learn to calculate the exact effective memory address from assembly code snippets.

• Wed–Fri: Pipelining and Cache are the two highest-weightage areas in COA. Calculate structural, data, and control hazards. Master cache size math (Tag bits, Index bits, Offset bits). Reminder: Secondary storage/disk math is deprecated in the latest curriculum—skip it to save time.

• Sat: Focus on Control Unit micro-operations and hardwired vs. microprogrammed design differences.

• Sunday: Take a COA subject test. Expect your score to be low initially; analyze the exact mathematical formulation you missed.

🗓️ Week 7 (Nov 23 – Nov 29): Databases (DBMS) & Computer Networks (CN)

• Syllabus Focus: Entity-Relationship Model, Relational Algebra, SQL; Functional Dependencies, Normalization (1NF to BCNF); IPv4 Routing, CIDR Subnetting, TCP Congestion Control, Sliding Window Protocols.

• AIR 70 Execution Strategy:

• Mon–Tue: Treat Normalization as discrete optimization—finding candidate keys using attribute closures. Master complex SQL queries involving GROUP BY, HAVING, and nested subqueries.

• Wed–Fri: For Networks, ignore dry theoretical layers. Focus heavily on CIDR subnet splitting calculations, TCP window size transitions during congestion, and Sliding Window Protocol efficiency math (Go-Back-N, Selective Repeat). Note: Old protocols like SMTP, DHCP, and ARP are out of the 2027 cycle.

• Sat: Wrap up Transactions and Concurrency Control (Conflict Serializability, 2PL).

• Sunday (Nov 30): Take a combined DBMS/CN subject test to wrap up the entire syllabus.

3 Non-Negotiable Tactics for AIR under 70

1. The "Never Guess" Rule for MSQs: Multiple Select Questions have no negative marking, but they require all correct options to be selected. Top rankers treat MSQs like four individual True/False questions. If you cannot mathematically prove an option is true, do not select it based on "intuition."

2. Use the Virtual Calculator Daily: Stop using your physical phone or scientific calculator starting today. Download the official GATE virtual calculator app or extension. Doing log, exponent, and matrix computations using the virtual interface takes practice; speed here saves 15 minutes on exam day.

3. The Error Classification Matrix: When reviewing your weekly tests, don't just write down the correction. Flag the error type:

- Type A (Conceptual): I didn't know the theory. (Requires 30 mins of review).
- Type B (Silly/Calculation): I wrote 2+3=6 or missed a minus sign. (Requires slower scratchpad work).
- Type C (Misread): I missed the word "NOT" or "INCORRECT" in the question statement. (Requires reading the question twice before touching the calculator).

The 3 Phases of the Mock Test Season

[Phase 1: Context Switching (Dec 1 - Dec 15)] —> [Phase 2: Peak Simulation (Dec 16 - Jan 20)] —> [Phase 3: Tapering & Calibration (Jan 21 - Exam)]

Phase 1: The Context-Switching Phase (December 1 – December 15)

Objective: Build the mental stamina to jump from a Data Structures pointer question to a Computer Networks subnetting calculation instantly.

• Action Plan: Do not jump straight into Full-Length Mocks (FLTs) yet. Write Multi-Subject Tests (e.g., OS + COA or Algorithms + Discrete Math).

• Frequency: Take one multi-subject test every single day.

• Daily Routine: Spend 3 hours taking the test, 2 hours tracking mistakes, and 4 hours deeply revising the specific sub-topics you got wrong.

Phase 2: Peak Simulation Phase (December 16 – January 20)

Objective: Build a bulletproof exam-taking strategy and eliminate psychological fatigue.

• Frequency: Take 1 Full-Length Mock Test (FLT) every 3 days (roughly 10–12 high-quality mocks in total).

• The Strict 9:30 AM Protocol: Always write your mocks during the exact official GATE time slots: 9:30 AM – 12:30 PM or 2:30 PM – 5:30 PM. Sit at a desk, use only the on-screen virtual calculator, and use unruled scratch pads. Your brain must recognize this window as peak performance time.

• The 4-Hour Post-Mock Analysis: Never close a mock test without completing this audit. Categorize every single lost mark into a tracker:

Phase 3: Tapering & Psychological Calibration (January 21 – Exam Day)

Objective: Solidify your strengths, prevent burnout, and tune your mind for precision.

• Stop Taking New Mocks: After January 20, stop writing new mock tests. A low score right before the exam will destroy your confidence.Y

• The PYQ Lockdown: Treat the official GATE 2024, 2025, and 2026 papers as your final holy grails. If you have already solved them, re-solve them completely to align your mindset with the official IISc/IIT question-framing style.

• Error Notebook Memorization: Read your cumulative Error Notebook from cover to cover every single morning.

The "AIR 70" Exam-Taking Strategy (The Round Method)

When writing your full-length mocks, divide your 3-hour window into three distinct rounds to maximize your score profile:

• **Round 1 (Minutes 0 to 60) — The Low-Hanging Fruit:**

• Start with General Aptitude (15 Marks). It is highly intuitive and provides an immediate confidence boost.

• Move to Technical 1-mark and 2-mark questions. Solve only the questions where the path to the answer is instantly obvious. If a problem looks like it will take more than 4 minutes, mark it for review and move on.

• **Round 2 (Minutes 60 to 140) — The Heavy Lifting:**

• Attack the complex, multi-step numerical questions, Dynamic Programming problems, and detailed System Software/COA questions.

• Treat Multiple Select Questions (MSQs) with extreme care. Evaluate each option as an independent True/False statement.

• **Round 3 (Minutes 140 to 180) — The Clean-Up & Verification:**

• Review the questions you marked for review.

• Verify your Numerical Answer Type (NAT) values. Re-read the rounding instructions (e.g., "round off to 2 decimal places" or "up to the nearest integer").

3 Rules to Break the 80-Mark Barrier

1. Isolate Your Top Score Drags: If your analysis shows you are consistently losing marks in a specific area (like COA Pipelining or OS Semaphores), pause your mock cycle for 24 hours. Go back to GATE Overflow and solve 40 consecutive raw questions on that exact micro-topic until it is completely neutralized.

2. The "Scratchpad Organization" Hack: Divide your rough sheet into numbered quadrants matching the question numbers. If you need to revisit Question 14 in Round 3, you should be able to instantly find your half-completed calculation instead of scanning random scribbles.

3. Master the "Skip" Instinct: To get an AIR under 70, knowing what to skip is just as critical as knowing what to solve. If an institutional examiner includes an incredibly lengthy, convoluted question designed to waste your time, skip it gracefully. Secure your 80 high-yield marks first.

To fine-tune this final stretch, which test series provider are you planning to use for your full-length mocks? I can also set up a task to track key upcoming milestones, such as when the GATE 2027 admit card is officially released or when mock analysis portals open.